CAS Correctness and Progress Analysis

EPAM COE Research Notes | Contribution: CAS Correctness and Progress Analysis

1. What is CAS?

CAS stands for **Compare-And-Swap**. It is an atomic operation used in concurrent programming. CAS checks whether a shared value is still equal to an expected value. If it is, CAS changes it to a new value. If it is not, the operation fails.

Concept: CAS(variable, expected, newValue) → If variable == expected, change it to newValue; otherwise, fail.

2. Why is CAS Needed?

When multiple threads update the same variable, a race condition can produce an incorrect result. For example, if two threads both read counter = 0 and then write 1, the final result becomes 1 instead of 2.

Without protection: Read 0 → Read 0 → Write 1 → Write 1 → Final = 1 ■

CAS makes the update conditional and atomic, so two threads cannot both successfully change the same old value.

3. How CAS Works

Suppose counter = 10. Thread 1 wants to change 10 to 11. It performs CAS(10, 11). If the counter is still 10, the CAS succeeds and the value becomes 11. If another thread has already changed it to 12, the CAS fails.

Success: expected = 10, actual = 10 → 10 becomes 11
Failure: expected = 10, actual = 12 → no update

4. CAS Retry Loop

CAS is often used inside a loop. A thread reads the current value, calculates the new value, tries CAS, and retries if another thread changed the value first.

Read value → Calculate new value → CAS → Success: finish | Failure: read again and retry

5. CAS Correctness

CAS is useful for correctness because the comparison and update happen as one atomic operation. Only one competing thread can successfully change a value from a particular expected state. Other threads detect the change and can retry using the latest value.

The successful CAS is often treated as the **linearization point**—the moment at which the operation logically takes effect.

6. ABA Problem

A special issue with CAS is the ABA problem. A thread reads A, another thread changes A → B → A, and the first thread sees A again. It may incorrectly assume that nothing changed. Algorithms can address this using version numbers or stamped/tagged references.

7. What is Progress?

Correctness is only one part of concurrent programming. We also need to ask whether the system continues to make progress when many threads are competing.

For example, if Thread 1's CAS fails but Thread 2 succeeds, Thread 1 did not progress, but the system as a whole did. This is the basic idea of **lock-free progress**.

8. Lock-Free vs Wait-Free

A **lock-free** algorithm guarantees that some thread will make progress, even if an individual thread repeatedly fails. A **wait-free** algorithm gives a stronger guarantee: every thread completes its operation within a bounded number of steps.

Property	Meaning
Lock-Free	System continues making progress; one thread may repeatedly retry.
Wait-Free	Every thread is guaranteed to complete within a bounded number of steps.

9. CAS Performance and Contention

CAS is non-blocking, but it is not automatically faster in every situation. If many threads repeatedly update the same variable, many CAS operations may fail. Repeated retries consume CPU time and can reduce performance.

Therefore, CAS-based designs should reduce unnecessary contention and use suitable data structures. Performance should be measured with benchmarks rather than assumed.

10. Simple Experiment

Create several threads and ask each thread to increment a shared counter many times. Compare a normal counter, a synchronized counter, and an AtomicInteger/CAS-based counter.

Implementation	Correctness	Typical behavior
Normal counter	May fail	Race condition possible
Synchronized/lock	Correct	Threads may block
Atomic/CAS	Correct when properly used	Non-blocking; retries possible

11. Key Findings

• CAS provides an atomic compare-and-update operation.
• A failed CAS usually means another thread changed the value first.
• Retry loops allow CAS-based algorithms to continue operating.
• The successful CAS can serve as the linearization point.
• The ABA problem must be considered in some algorithms.
• Lock-free means system-wide progress; wait-free is a stronger guarantee.
• High contention can cause repeated CAS failures and lower performance.

12. Conclusion

CAS is a fundamental building block for lock-free concurrent programming. It helps maintain correctness without relying on traditional locks. However, a good concurrent algorithm must be evaluated for **correctness, progress, and performance**. CAS provides the foundation, while careful algorithm and data-structure design determines whether the final system is reliable and efficient.